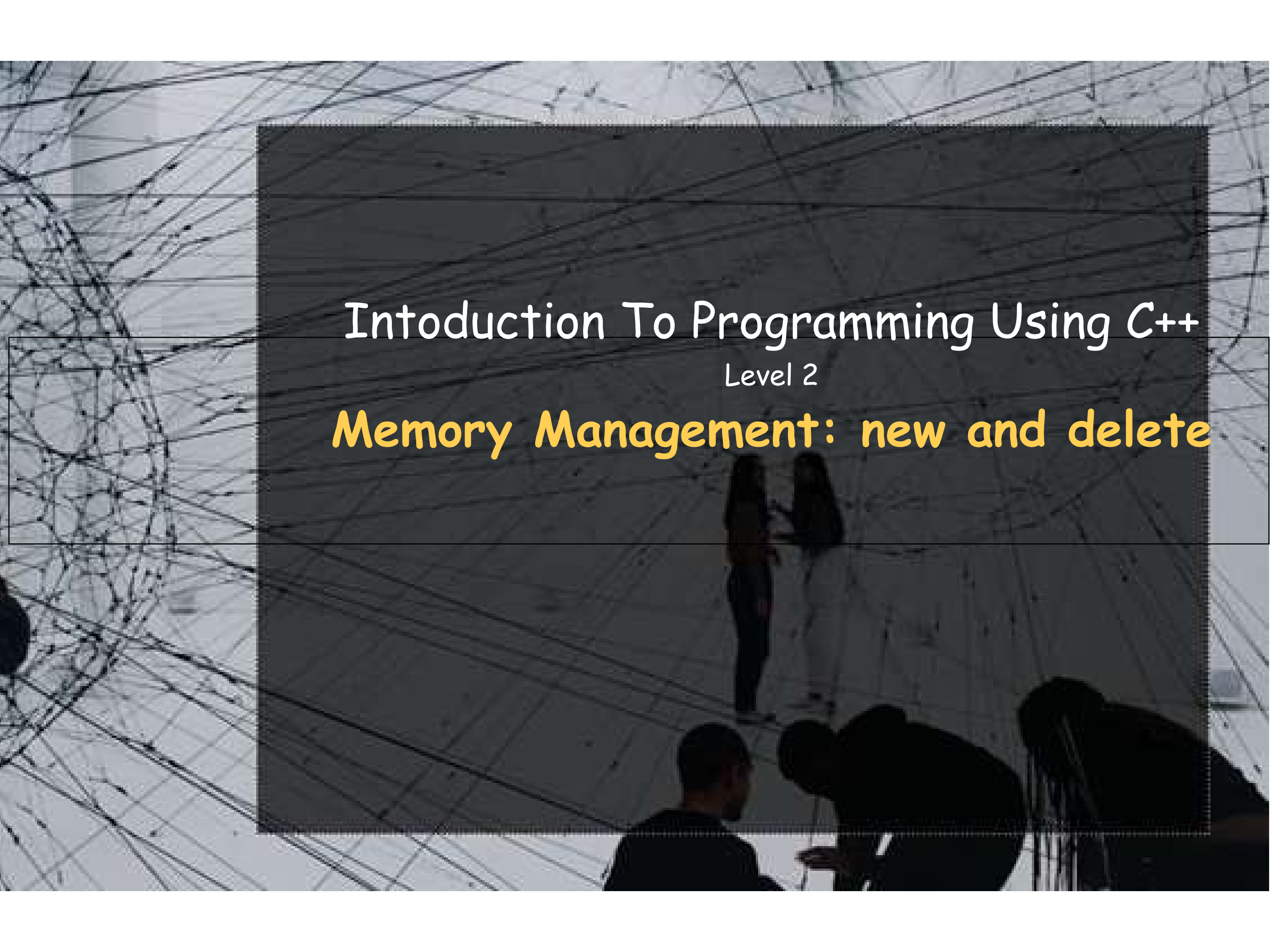


The background is a blurred image of a computer monitor. It shows a dark-themed code editor with some blue and green highlights. At the bottom, a Windows taskbar is visible with various application icons like Chrome, Photoshop, and others.

Introduction To Programming Using C++ Level 2

Memory Management: new and delete



Introduction To Programming Using C++

Level 2

Memory Management: new and delete

🔍 Definition

■ Dynamic Memory Allocation

■ Dynamic Memory Allocation means allocating memory during run-time instead of having the memory size fixed at compile-time.

■ In C++, we use the following keywords:

■ `new` → to allocate memory.

■ `delete` → to deallocate memory.

Explanation

`new`

 When using `new`, the program allocates memory dynamically and returns its address to a pointer.

 The allocated memory is typically taken from the Heap.

`delete`

 After finishing with dynamically allocated memory, we should release it using `delete`.

 This helps prevent memory leaks.

`nullptr`

 After deleting the memory, it is a good practice to set the pointer to `nullptr`.

 This helps prevent accidentally using the pointer as if it still pointed to a valid object.

Procedures

1. Create pointer variables.
2. Use ``new`` to allocate memory.
3. Store the returned address in the pointer.
4. Use ``*pointer`` to access the allocated memory.
5. Assign a value to the allocated memory.
6. Use the value when needed.
7. Use ``delete`` to release the memory.
8. Set the pointer to ``nullptr``.

💡 Example

```
int* ptrX;  
float* ptrY;  
ptrX = new int;  
ptrY = new float;  
*ptrX = 45;  
*ptrY = 58.35f;  
cout << *ptrX << endl;    // 45  
cout << *ptrY << endl;    // 58.35  
delete ptrX;  
delete ptrY;  
ptrX = nullptr;  
ptrY = nullptr;
```

➡ Result

45

58.35

Note

- ◆ ``new`` dynamically allocates memory and returns its address.
- ◆ ``delete`` releases memory that was allocated with ``new``.
- ◆ After ``delete``, setting the pointer to ``nullptr`` is a good safety practice.
- ◆ In modern C++, manually using ``new`` and ``delete`` is usually avoided when smart pointers can manage the lifetime automatically.
- ◆ Smart Pointers such as ``std::unique_ptr`` and ``std::shared_ptr`` are commonly preferred for managing dynamic objects.

⊘ Warning

- ◆ Do not use `delete` twice on the same pointer.

```
delete ptrX;
```

```
delete ptrX; // ✗
```

- ◆ Forgetting to release dynamically allocated memory can cause a memory leak.
- ◆ Do not dereference a pointer after the memory it points to has been deleted.
- ◆ Setting the pointer to `nullptr` after `delete` helps prevent accidental use of the invalid pointer.



Comparison

▨ `new`

- ✓ Allocates memory dynamically.
- ✓ Returns the address of the allocated memory.
- ✓ The address is usually stored in a pointer.
- ✓ Example: `ptr = new int;`

▩ `delete`

- ✓ Releases memory previously allocated with `new``.
- ✓ Should be used when the dynamically allocated object is no longer needed.
- ✓ Example: `delete ptr;`

✦ Summary

- ✓ ``new`` is used to allocate dynamic memory.
- ✓ ``delete`` is used to release memory allocated with ``new``.
- ✓ Dynamically allocated memory is typically located in the Heap.
- ✓ A pointer stores the address of the allocated memory.
- ✓ ``*ptr`` is used to access the value stored in the allocated memory.
- ✓ Set the pointer to ``nullptr`` after ``delete`` as a safety practice.
- ✓ Smart Pointers such as ``std::unique_ptr`` and ``std::shared_ptr`` are preferred in many modern C++ situations.

Thank you 😊